

DISEÑO DE SISTEMAS DE INFORMACIÓN

“METAMAPA”

JUSTIFICACIONES DE DISEÑO

Trabajo Práctico Anual Integrador
-2025-

Grupo N°5



ENTREGA 5: Arquitectura Web MVC y Maquetado de Interfaz de Usuario

Arquitectura de Sesiones Mixtas

Nuestro sistema implementa una arquitectura de sesiones mixtas, combinando manejo de sesión tradicional (basado en cookies) con autenticación moderna mediante tokens JWT. Esto permite mantener una experiencia de usuario fluida en el cliente web mientras se conserva la independencia y seguridad de los microservicios.

Cliente Liviano Desacoplado (Frontend con Thymeleaf)

Nuestro frontend fue desarrollado como un Cliente Liviano Desacoplado, implementado con Thymeleaf y Spring MVC.

Esto implica que:

- Las vistas HTML son renderizadas del lado del servidor (server-side rendering).
- El cliente mantiene sesiones stateful mediante cookies de sesión.
- El usuario interactúa con formularios tradicionales (por ejemplo, el de login), y las redirecciones se manejan dentro del mismo flujo del servidor frontend.
- Se manejan tokens de autorización desde nuestro Servidor Frontend hacia nuestro Servidor de Aplicación (donde reside la lógica de negocio).

Manejo de Sesiones y Tokens

Mientras que el cliente liviano (donde residen las vistas) es stateful y maneja sesiones de usuarios, todos los microservicios que fueron diseñados aparte son stateless y manejan tokens para la autorización.

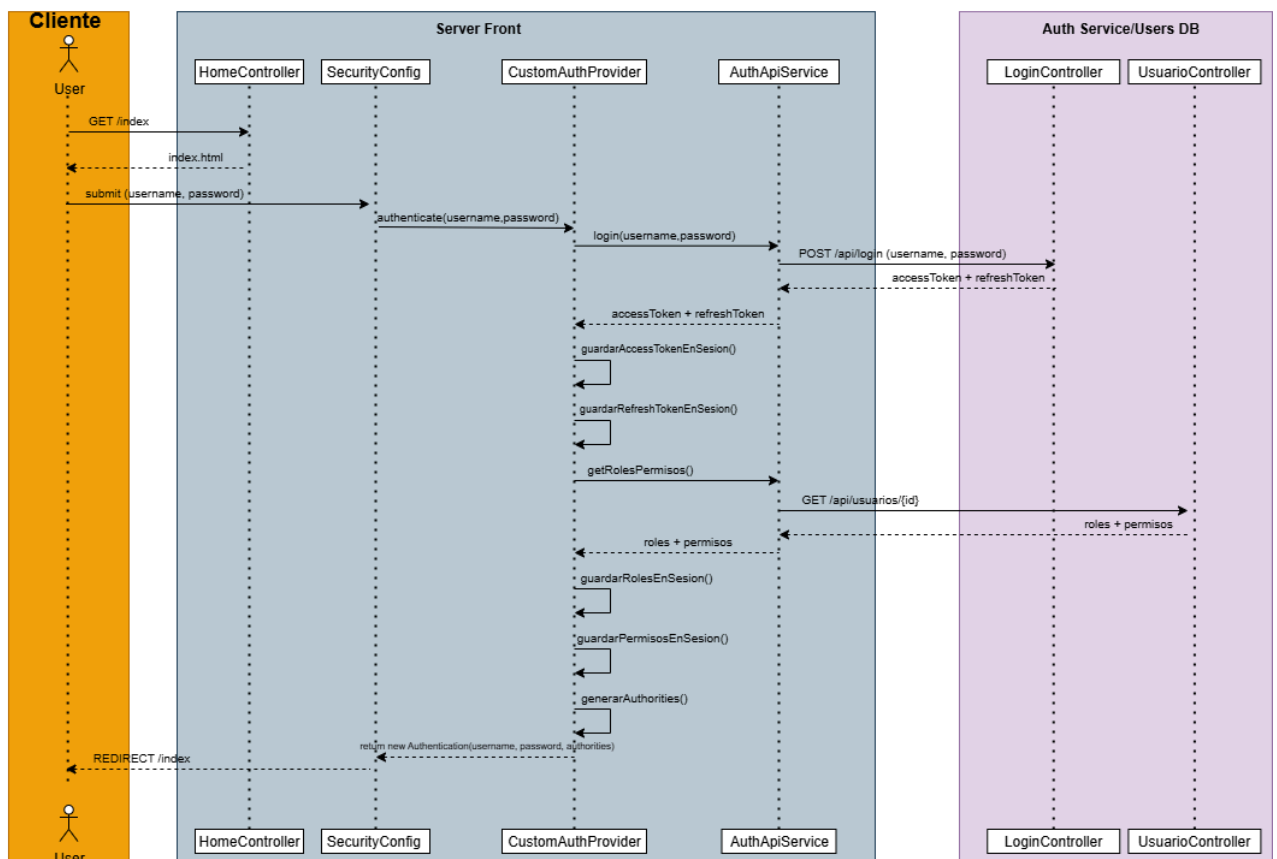
El Servidor Frontend cumple dos roles principales:

1. Gestión de sesión con el usuario final mediante cookies (HttpSession).
2. Comunicación segura con el servicio de autenticación usando tokens JWT.

El flujo es el siguiente:

1. El usuario accede a /login y envía sus credenciales mediante un formulario.
2. El SecurityConfig delega la autenticación al CustomAuthProvider.
3. El CustomAuthProvider, a través del AuthApiService, realiza un POST al endpoint externo {authservice.url}/api/auth con las credenciales.
4. El servicio devuelve un accessToken y un refreshToken.

5. El frontend guarda estos tokens en la sesión del usuario (no en el navegador), manteniendo la cookie de sesión activa.
6. Con el accessToken, el frontend consulta los roles y permisos del usuario: GET {url}/api/auth/user/roles-permiso
7. Los roles y permisos se guardan en la sesión y se transforman en GrantedAuthorities para Spring Security.
8. Si el token expira, el frontend lo renueva mediante: POST {url}/api/auth/refresh



Auth Service (Microservicio Stateless)

El servicio de autenticación es un microservicio independiente y stateless, responsable de la autenticación y gestión de usuarios.

Se encarga de:

- Emitir accessTokens y refreshTokens.
- Validar credenciales y sesiones.
- Exponer endpoints de consulta de roles y permisos.
- Exponer endpoints de creación, eliminación y actualización de usuarios.

Además de la emisión y validación de tokens, el servicio de autenticación se encarga de persistir los usuarios en la base de datos.

Endpoints principales:

- POST /api/auth → autentica y genera access/refresh tokens
- POST /api/auth/refresh → renueva el access token usando el refresh token
- GET /api/auth/user/roles-permisos → obtiene roles y permisos del usuario
- GET /api/usuarios/{id} → consulta datos del usuario (con token)
- POST /api/usuarios → crea nuevo usuario
- PUT /api/usuarios/{id} → actualiza usuario existente (con token)
- DELETE /api/usuarios/{id} → elimina usuario (con token)

Auth Manager (Securización de los servicios)

Todos los microservicios que fueron diseñados aparte son stateless y manejan tokens para la autorización de peticiones securizadas tales como alterar Colecciones o permisos de administrador.

Consideramos que no era necesario asegurar los endpoints de Fuente Proxy y Fuente Estática, ya que se trata de *APIs públicas* de consulta de datasets a disposición de cualquier usuario. No existe la posibilidad de modificación. Es únicamente de consulta (HTTP GET).

Servicios como Fuente Dinámica y Servicio Agregador sí requieren autorización, ya que implican la alteración de hechos o colecciones. Esto requiere de permisos tanto para el Contribuyente o Administrador, según corresponda.

Las rutas de estos solicitan un Bearer token para autorizar, sin embargo ningún servicio tiene la clave. El encargado de la autorización es el Servicio de Autenticación. Como solución, proponemos validación remota con una clase AuthManager.

El AuthManager, cuando recibe un request con Bearer token, llama al servicio Autenticación (por HTTP) para validar el token.

Ventajas:

- Centraliza la validación.
- Cada servicio no necesita la clave secreta.

Desventaja:

- Cada request protegida hace una llamada extra al servicio de autenticación.
- Mayor latencia
- Acoplamiento al Servicio de Autenticación.

El servicio de autenticación valida en la ruta:

`http://localhost:8087/api/auth/validation`

Paginación

Se implementó paginación al traer colecciones, hechos y solicitudes. Esto gracias a Pageable de JPA.

El json output queda de la siguiente manera:

```
{
  "content": [
    {
      "id": 1,
      "idHecho": null,
      "motivoBorrado": "gratis",
      "estado": "RECHAZADA_POR_SPAM"
    }
  ],
  "page": 0,
  "limit": 1,
  "totalElements": 14,
  "totalPages": 14,
  "hasNext": true,
  "hasPrevious": false
}
```